INFORMATIONSBLATT

IB01 – Entscheidungen mit if, else und elseif

Dynamische Webseiten mit PHP · LS04 | 2BKI2 · Berufskolleg Technische Assistenz · Schuljahr 2026/2027

🎯 Darum geht es

Sie lassen PHP abhängig von einer Bedingung unterschiedlichen Code ausführen, formulieren Bedingungen mit Vergleichsoperatoren und kennen den Unterschied zwischen `==` und `===`.

1 | Die einfache Verzweigung: if

Mit `if` (englisch „wenn“) wird ein Codeblock nur ausgeführt, wenn die Bedingung in den runden Klammern **wahr** ist. Der Block steht in geschweiften Klammern.

if — beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>if</title></head>
<body>

<?php
  $personen = 14;

  if ($personen > 12) {
    echo "<p>So viele Plätze haben wir leider nicht.</p>";
  }
  echo "<p>Diese Zeile erscheint immer.</p>";
?>

</body>
</html>
```

👁 SO SIEHT ES IM BROWSER AUS



Abbildung 1: Die Bedingung ist wahr (14 > 12), also erscheint der erste Satz.

💡 EINRÜCKEN

Den Inhalt eines Blocks rückt man zwei oder vier Leerzeichen ein. PHP ist das egal – Menschen erkennen so aber sofort, was zusammengehört. Visual Studio Code rückt beim Tippen automatisch ein.

2 | Zweiseitig: if und else

`else` („sonst“) enthält den Code für den Fall, dass die Bedingung **falsch** ist. Genau einer der beiden Blöcke wird ausgeführt – nie beide, nie keiner.

if und else beispiel.php

```
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>if else</title></head>
<body>

<?php
  $platz = "aussen";

  if ($platz == "aussen") {
    echo "<p>Sie sitzen draußen unter der Markise.</p>";
  } else {
    echo "<p>Sie sitzen drinnen am Fenster.</p>";
  }
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS

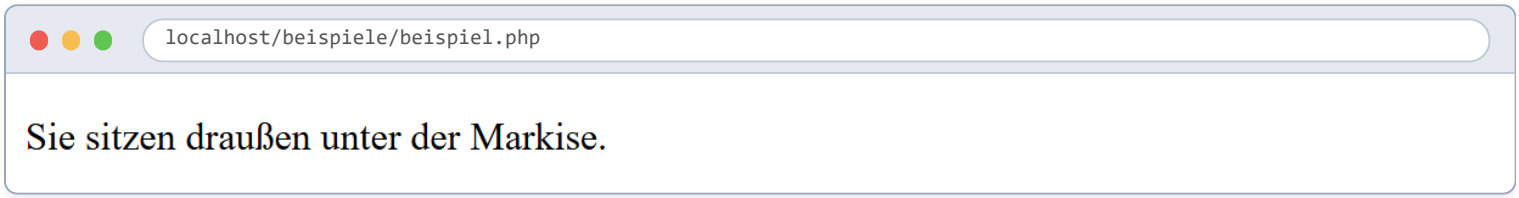


Abbildung 2: Da $platz den Wert "aussen" hat, wird der if-Block ausgeführt.

3 | Mehrseitig: elseif

Gibt es mehr als zwei Fälle, prüft `elseif` („sonst wenn“) eine weitere Bedingung. PHP geht die Bedingungen **von oben nach unten** durch und führt nur den **ersten** zutreffenden Block aus.

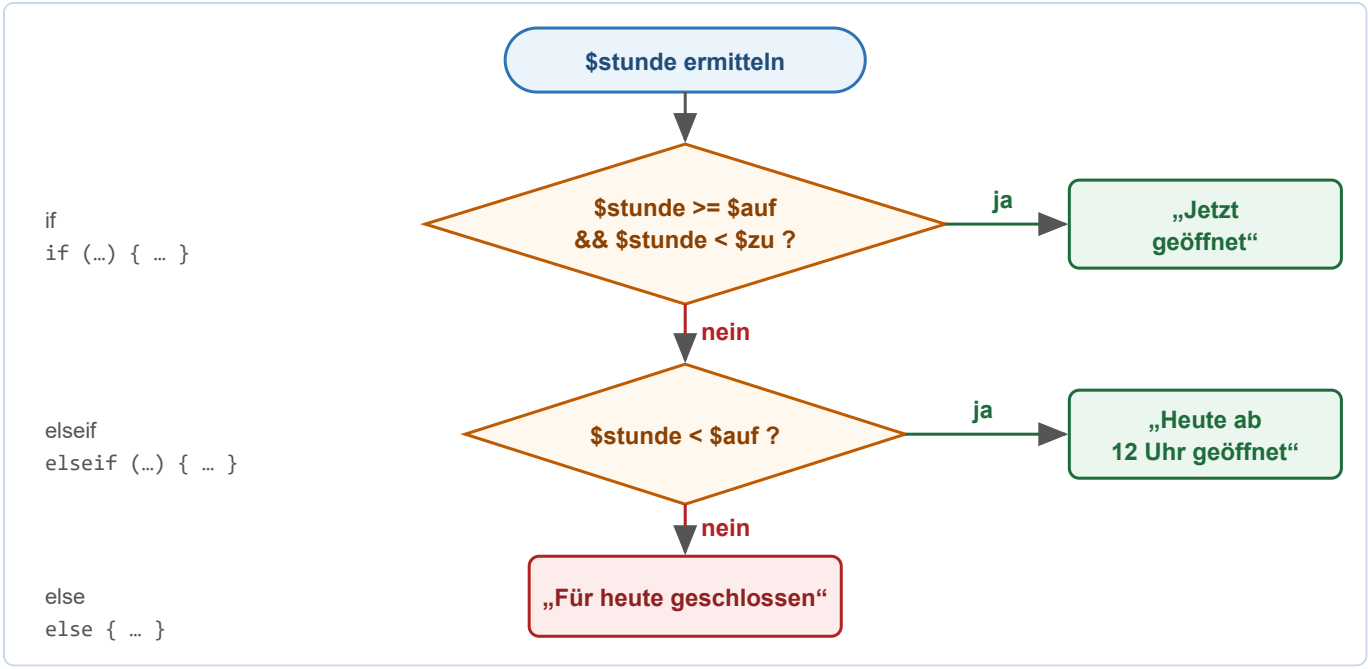


Abbildung 3: Der Öffnungsstatus als Ablauf: Nur ein Weg wird durchlaufen.

if, elseif und else beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>elseif</title></head>
<body>

<?php
  $kugeln = 4;

  if ($kugeln == 1) {
    echo "<p>Eine Kugel – der Klassiker.</p>";
  } elseif ($kugeln <= 3) {
    echo "<p>$kugeln Kugeln passen in die Waffel.</p>";
  } else {
    echo "<p>$kugeln Kugeln? Dafür nehmen wir lieber einen Becher.</p>";
  }
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS

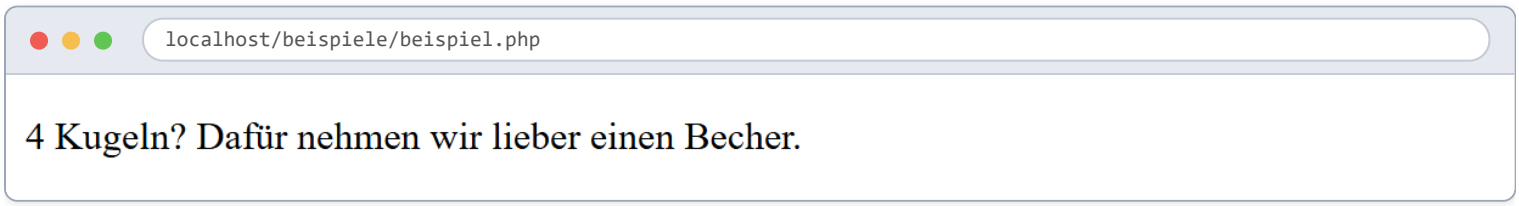


Abbildung 4: Die Bedingungen werden der Reihe nach geprüft; 4 ist weder 1 noch höchstens 3.

4 | Vergleichsoperatoren

Operator	Bedeutung	Beispiel (wahr)
`==`	gleich (Wert)	`4 == "4"`
`===`	identisch (Wert **und** Typ)	`4 === 4`
`!=`	ungleich	`$platz != "innen"`
`!==`	nicht identisch	`4 !== "4"`
`<` `>`	kleiner, größer	`3 < 12`
`<=` `>=`	kleiner/größer oder gleich	`12 <= 12`

== und === beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>== und ===</title></head>
<body>

<pre>
<?php
  var_dump(4 == "4");     // gleicher Wert?
  var_dump(4 === "4");    // gleicher Wert UND gleicher Typ?
  var_dump(4 === 4);
?>
</pre>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS

localhost/beispiele/beispiel.php

```
bool(true)
bool(false)
bool(true)
```

Abbildung 5: == vergleicht nur den Wert, === zusätzlich den Datentyp.

⚠️ = IST NICHT ==

`if ($platz = "aussen")` **weist zu**, statt zu vergleichen – die Bedingung ist dann immer wahr, und `$platz` wurde nebenbei überschrieben. Ein klassischer Fehler, der keine Fehlermeldung erzeugt. Zum Vergleichen immer `==` oder `===`.

💡 WANN ===?

Formulardaten sind Strings. `$_POST['personen'] === 4` ist deshalb **immer falsch** – der String "4" ist nicht identisch mit der Zahl 4. Entweder vorher mit `(int)` umwandeln oder `==` verwenden. Bei Texten ist `===` die sichere Wahl: `$name === ""`.

5 | Texte vergleichen

Texte vergleicht PHP Zeichen für Zeichen; Groß- und Kleinschreibung zählt: `"Mango" == "mango"` ist falsch. Praktisch: Datumsangaben im Format `JJJJ-MM-TT` lassen sich direkt mit `<` und `>` vergleichen, weil die größte Einheit vorne steht:

Datum mit dem heutigen Tag vergleichen

```
if ($datum < date("Y-m-d")) {
  echo "Dieses Datum liegt in der Vergangenheit.";
}
```